Quantum π: Estimating π Using Quantum Randomness

A Project Report prepared as an independent study and presented out of personal academic interest in the field of Quantum Computing.

Submitted by:

Nayan Bhandari[1]
REG No: 011-1111-0248-24

[1]Department of Physics, A.J.C. Bose College

Under the guidance of:

DR. BIBEKANANDA MAJI[2]
DR. KAMALIKA HAJRA[2]

[2]Assistant Proffessor, Department of physics, A.J.C. Bose College

Department of Physics

Acharya Jagadish Chandra Bose College
University of Calcutta
1/1B, A.J.C. Bose Road, Kolkata-700020

Contents

1 Abstract **2**

2 Introduction **2**
2.1 The Classical Computational Benchmark . . . 2
2.2 The Quantum Paradigm Shift . . . 2

3 The Problem Statement **3**
3.1 The Problem . . . 3
3.2 The Proposed Quantum Solution . . . 3

4 Theoretical Frameworks **4**
4.1 Classical VS Quantum Approach in Monte Carlo Simulation . . . 4
4.1.1 Classical Monte Carlo . . . 4
4.1.2 The Quantum Approach - this project based only on Quantum Randomness . . . 4
4.2 Mathematical Interpretation . . . 4
4.3 The Quantum interpretation . . . 5

5 Methodology and Implementation **8**
5.1 Software Stack . . . 8
5.2 Circuit Design . . . 8
5.3 Implementation Walkthrough . . . 8
5.3.1 Importing Libraries . . . 8
5.3.2 Binary Fraction Conversion . . . 8
5.3.3 The Quantum Random Number Generator (QRNG) . . . 9
5.3.4 Monte Carlo Execution . . . 9
5.3.5 Estimation and Visualization . . . 9

6 Result and Analysis **11**
6.1 Experimental Data Summary . . . 11
6.2 Analysis of Graphical Outputs . . . 12
6.2.1 A. Error vs. Total Points (Interpolated Analysis) . . . 12
6.2.2 B. Time Taken vs. Total Points . . . 12
6.2.3 C. Error vs. Time Taken (Efficiency Trade-off) . . . 13

7 Discussion **14**
7.1 Interpretation of Results: . . . 14
7.2 Hardware vs. Simulator: Ideal Results vs. Physical Quantum Hardware . . . 14
7.3 Error Mitigation: . . . 14

8 Conclusion and Future Work **15**
8.1 Summary . . . 15
8.2 Limitations . . . 15
8.3 Future Work . . . 15

1 Abstract

This project implements a quantum-enhanced Monte Carlo simulation to estimate the value of π (pi) $\approx$ 3.14159. By generating truly random coordinates using quantum superposition and measurement via the Qiskit framework, it simulates "dart throws" onto a unit square containing a quarter-circle. The proportion of darts landing inside the circle, scaled by 4, yields an approximation of π. Unlike classical methods relying on pseudo-random number generators (PRNGs), this approach harnesses quantum indeterminacy for unbiased randomness, making it suitable for applications requiring high-fidelity stochastic sampling, such as financial modeling or scientific simulations.

Key features:

- **Quantum Random Number Generation (QRNG)**: Uses Hadamard gates on multiple qubits to produce uniform binary strings.
- **Configurable Parameters**: User-defined number of simulation points (NUMPOINT) and precision (PRECISION, i.e., number of qubits).
- **Visualization**: Matplotlib-based scatter plot showing points colored by circle membership (green inside, red outside).
- **Error Analysis**: Computes absolute error against NumPy's high-precision π.

The code, dated December 25, 2025, is a self-contained Python script ($\sim$70 lines) that runs on classical hardware via Qiskit's AerSimulator,requiring no quantum device access.

2 Introduction

The mathematical constant pi (π) has captivated human curiosity for millennia. From the geometric approximations of ancient Babylonians and Archimedes to the infinite series discovered by Madhava and Leibniz, the quest to calculate pi with increasing precision has consistently driven mathematical innovation. Historically, the pursuit of pi was not merely about knowing its exact value which is practically useful only to a few dozen decimal places in physics and engineering but about developing new mathematical techniques to push the boundaries of human knowledge.

2.1 The Classical Computational Benchmark

With the advent of the digital age, pi took on a new role: the ultimate computational benchmark. Early mainframes like the ENIAC were tasked with calculating pi to test their processing limits. Today, supercomputers churn out trillions of digits of pi to stress-test hardware architecture, memory optimization, and classical algorithms like the Fast Fourier Transform. In classical computer science, pi is a fundamental yardstick for computational power and stability.

2.2 The Quantum Paradigm Shift

As we enter the era of quantum computing, the calculation of pi is once again serving as a frontier benchmark, but the objective has fundamentally shifted. Quantum computers rely on the principles of superposition, entanglement, and interference, representing a complete departure from classical deterministic bits. Therefore, the significance of calculating pi on a quantum computer is not to discover more digits, but to validate the quantum computing paradigm itself. Classical computers can already calculate trillions of digits flawlessly; quantum computers, in their current Noisy Intermediate-Scale Quantum (NISQ) era, struggle to accurately calculate the first few.

Computational Significance of the Quantum Approach :

Simulating the value of pi using quantum mechanics holds profound significance for several reasons:

- **Algorithmic Validation :**

  Approximating pi serves as a perfect proof-of-concept for foundational quantum algorithms. Whether utilizing Quantum Phase Estimation (QPE) to extract phase information or applying Quantum Monte Carlo methods (Amplitude Estimation) to evaluate integrals, simulating pi demonstrates that these theoretical quantum algorithms function correctly in practice.

- **Hardware Benchmarking :**

  Because the mathematical value of pi is perfectly known, it acts as an ideal diagnostic tool for quantum hardware. By comparing the quantum-simulated value to the true value, researchers can precisely measure the impact of quantum noise, gate errors, and qubit decoherence. It provides a clear, quantitative metric of a quantum processing unit's (QPU) fidelity.

- **The Quantum Advantages :**

  While calculating pi does not currently offer a "quantum advantage" (where a quantum computer solves a problem a classical computer cannot), the algorithms used to simulate it are the exact same algorithmic primitives that will eventually power quantum chemistry, cryptography, and complex financial modelling.

This project explores the intersection of this ancient mathematical constant and modern quantum technology. By implementing a quantum circuit to simulate the value of pi, this report will outline the theoretical mechanics of the chosen quantum algorithm, detail its software implementation, and analyse the results to understand both the power and the current limitations of quantum computation.

3 The Problem Statement

3.1 The Problem

Classical computers are deterministic machines; therefore, they cannot generate true random numbers. Instead, they rely on pseudo-random number generators (PRNGs)mathematical algorithms that produce sequences that merely appear random. Over millions of iterations, PRNGs can exhibit periodicity and hidden correlations, which can introduce statistical biases and limit the ultimate accuracy of large-scale probabilistic simulations.

3.2 The Proposed Quantum Solution

Unlike classical systems, quantum mechanics is fundamentally probabilistic. When a qubit is placed into a state of uniform superposition using a Hadamard gate and then measured, the resulting state collapse yields a strictly unpredictable, truly random output.

Project Objective: This project seeks to replace the classical PRNG bottleneck in Monte Carlo simulations with a Quantum Random Number Generator (QRNG). Specifically, the objectives are to:

- 1. Construct a quantum circuit using Qiskit that applies Hadamard gates to generate uniform superpositions, yielding true random bitstrings.
- 2. Translate these quantum-generated bitstrings into decimal coordinate values (x, y) mapped to a 2D plane.
- 3. Execute a Monte Carlo estimation of the constant π by determining the ratio of quantum-generated points falling inside a bounded circle versus a bounding square.
- 4. Analyze the error rate between the simulated value and the actual value of π to evaluate the viability of using near-term quantum hardware/simulators as entropy sources for statistical modeling.

4 Theoretical Frameworks

4.1 Classical VS Quantum Approach in Monte Carlo Simulation

4.1.1 Classical Monte Carlo

The classical Monte Carlo method relies on the law of large numbers to approximate complex values, such as π, through repeated random sampling.

• **Mechanism:** It generates N random coordinate pairs (x, y) and tests how many fall within a specific boundary (e.g., a circle inscribed in a square).

• **The Limitation:** Classical computers are deterministic and must rely on Pseudo-Random Number Generators (PRNGs). PRNGs are mathematical formulas that simulate randomness but ultimately contain hidden periodicities and correlations. Over millions of iterations, these deterministic artifacts can introduce statistical biases into high-precision physics simulations.

• **Scaling:** The standard error in a classical Monte Carlo simulation scales at a rate of $O(1/\sqrt{N})$. To improve the accuracy by a factor of 10, one must generate 100 times as many samples.

4.1.2 The Quantum Approach - this project based only on Quantum Randomness

The quantum approach includes the Hadamard gate to generate superposition of Qubits. The method is exactly same as the classical method of finding π value.

• **Mechanism:** Rather than altering the underlying Monte Carlo math, this approach replaces the classical PRNG with a Quantum Random Number Generator (QRNG). Qubits are initialized into a state of uniform superposition using Hadamard gates. According to the Born rule, measuring this superposition forces a purely probabilistic wave-function collapse, yielding a fundamentally unpredictable, true random bit.

• **Advantage:** While the mathematical convergence still scales at $O(1/\sqrt{N})$, it completely eliminates the periodic biases inherent to classical PRNGs, providing a mathematically "pure" entropy source for the statistical sampling.

4.2 Mathematical Interpretation

To extract the value of π from the quantum-generated data, the simulation relies on a fundamental geometric relationship between a circle and a bounding square.

The Geometric Model The simulation mathematically constructs a square on a 2D Cartesian plane. For simplicity, assume a square with side lengths of $2r$. Inscribed perfectly within this square is a circle with a radius of r. The area of the bounding square is $A_{\text{square}} = (2r)^2 = 4r^2$. The area of the inscribed circle is $A_{\text{circle}} = \pi r^2$. **The Area Ratio** By calculating the ratio of the circle's area to the square's area, the r^2

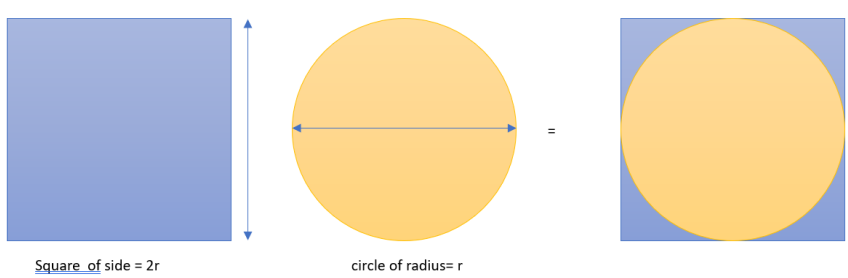

Figure 1: Visual representation of the dart board.

terms cancel out, mathematically isolating π:

$$\frac{A_{\text{circle}}}{A_{\text{square}}} = \frac{\pi r^2}{4r^2} = \frac{\pi}{4}$$

Applying the Quantum Data The quantum simulator acts as an entropy source, generating independent random variables that are translated into (x, y) coordinate pairs. These points are mapped uniformly across the area of the square. In the classical post-processing phase, the distance formula is applied to each coordinate pair to determine if it falls within the boundary of the inscribed circle. Assuming the circle is centered at the origin $(0, 0)$, a point is inside if:

$$x^2 + y^2 \leq r^2$$

Because the quantum-generated coordinates are uniformly distributed, the probability that any given point lands inside the circle is strictly equal to the ratio of their areas. Therefore, by dividing the total number of points that land inside the circle (N_{inside}) by the total number of points generated overall (N_{total}), we can approximate the probability and solve for π:

$$\frac{N_{\text{inside}}}{N_{\text{total}}} \approx \frac{\pi}{4}$$

$$\pi \approx 4 \times \left(\frac{N_{\text{inside}}}{N_{\text{total}}} \right)$$

4.3 The Quantum interpretation

The Qubit: A Geometric and Algebraic Object

A classical bit is a switch — it can only be **off** (0) or **on** (1). A qubit is a *direction in a two-dimensional complex vector space*. Its state is written as a unit vector:

$$|\psi\rangle = \alpha|0\rangle + \beta|1\rangle, \quad \alpha, \beta \in C, \quad |\alpha|^2 + |\beta|^2 = 1$$

The notation $|\cdot\rangle$ is **Dirac bra-ket notation** a standard shorthand for column vectors in quantum mechanics. The computational basis vectors are:

$$|0\rangle = \begin{pmatrix} 1 \\ 0 \end{pmatrix}, \quad |1\rangle = \begin{pmatrix} 0 \\ 1 \end{pmatrix}$$

So the full state $|\psi\rangle$ is literally:

$$|\psi\rangle = \alpha \begin{pmatrix} 1 \\ 0 \end{pmatrix} + \beta \begin{pmatrix} 0 \\ 1 \end{pmatrix} = \begin{pmatrix} \alpha \\ \beta \end{pmatrix}$$

This state is not "secretly 0 or 1 but we don't know which". It is a genuine superposition both outcomes exist simultaneously with amplitude α and β respectively, until measurement forces a collapse.

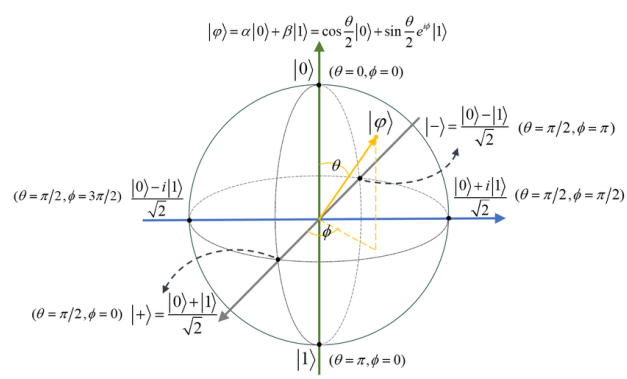

Figure 2: Geometric and algebraic representation of a Qubit.

Every qubit state satisfying the normalisation constraint lives on the surface of a unit sphere in 3D space called the **Bloch sphere**, the most important geometric tool for visualising single-qubit states.

The state can be parameterised using two real angles $\theta \in [0, \pi]$ and $\phi \in [0, 2\pi)$:

$$|\psi\rangle = \cos\frac{\theta}{2}|0\rangle + e^{i\phi}\sin\frac{\theta}{2}|1\rangle$$

The Hadamard Gate: Engineering Equal Superposition

The function applies a **Hadamard gate** H to each qubit. This is a single-qubit unitary transformation with the matrix representation:

$$H = \frac{1}{\sqrt{2}}\begin{pmatrix} 1 & 1 \\ 1 & -1 \end{pmatrix}$$

Applied to the computational basis states:

$$H|0\rangle = \frac{1}{\sqrt{2}}(|0\rangle + |1\rangle) \qquad H|1\rangle = \frac{1}{\sqrt{2}}(|0\rangle - |1\rangle)$$

Since every qubit is initialised to $|0\rangle$, after the Hadamard gate each qubit is in the state $\frac{1}{\sqrt{2}}|0\rangle + \frac{1}{\sqrt{2}}|1\rangle$, giving equal measurement probabilities:

$$P(\text{measure } 0) = \left|\frac{1}{\sqrt{2}}\right|^2 = \frac{1}{2}, \qquad P(\text{measure } 1) = \left|\frac{1}{\sqrt{2}}\right|^2 = \frac{1}{2}$$

This is the quantum analogue of a perfectly fair coin toss but it is *physically* rather than algorithmically guaranteed.

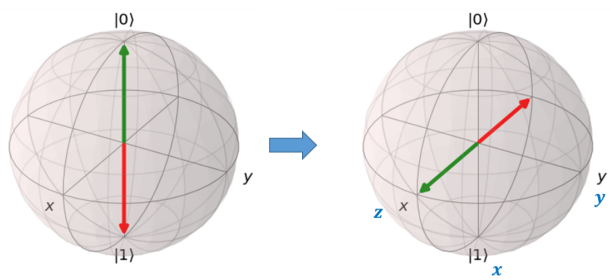

Figure 3: Geometric representation of Hadamard gate.

For the n-Qubit Register and Tensor Products

When the algorithm runs, it applies the Hadamard gate to every qubit in the register *simultaneously*. Mathematically this is the **tensor product** of n individual Hadamard gates:

$$H^{\otimes n} = H \otimes H \otimes \cdots \otimes H \quad (n \text{ times, where n correspond to precision})$$

For $n = 2$, the combined gate is the 4×4 matrix:

$$H^{\otimes 2} = H \otimes H = \frac{1}{2}\begin{pmatrix} 1 & 1 & 1 & 1 \\ 1 & -1 & 1 & -1 \\ 1 & 1 & -1 & -1 \\ 1 & -1 & -1 & 1 \end{pmatrix}$$

Applied to the initial two-qubit state $|00\rangle$, the result is:

$$H^{\otimes 2}|00\rangle = \frac{1}{2}(|00\rangle + |01\rangle + |10\rangle + |11\rangle)$$

Each of the four basis states now carries equal amplitude $\frac{1}{2}$ and equal probability $\frac{1}{4}$. For n qubits this generalises to a uniform superposition over all 2^n computational basis states, which is the mathematical source of the uniform random bitstrings the code uses to generate its coordinates.

Converting Bitstrings to Uniform Coordinates

The measured bitstring $b_1 b_2 \cdots b_n$ (where each $b_i \in \{0, 1\}$) is converted to a decimal coordinate by using the **binary fraction expansion**:

$$x = \sum_{i=1}^{n} \frac{b_i}{2^i} = \frac{b_1}{2} + \frac{b_2}{4} + \frac{b_3}{8} + \cdots + \frac{b_n}{2^n}$$

This maps the bitstring to a value in the half-open interval $[0, 1)$, specifically to one of 2^n uniformly spaced points on the grid $\{0, \frac{1}{2^n}, \frac{2}{2^n}, \ldots, \frac{2^n-1}{2^n}\}$.

Two independent circuits are run per dart one for x, one for y producing a point $(x, y) \in [0, 1)^2$. Because the quantum measurement is uniformly distributed, the resulting 2D point is a **discrete uniform sample** from a $2^n \times 2^n$ grid of positions inside the unit square.

Precision and resolution. The precision parameter n directly controls the resolution of the sampling grid. With n qubits per axis, the spacing between adjacent sample points is $\Delta = 2^{-n}$, and the total number of distinct coordinate pairs is 2^{2n}.

5 Methodology and Implementation

The computational core of this project uses a hybrid quantum-classical programming paradigm.

5.1 Software Stack

- **Programming Language:** Python 3
- **Quantum SDK:** `qiskit`, used to construct, transpile, and execute the quantum circuits.
- **Simulation Backend:** `qiskit_aer` (specifically `AerSimulator`) locally emulates the behavior of a quantum processor.
- **Classical Libraries:** `NumPy` for mathematical constants and `Matplotlib` for the visual rendering of the dart distribution.

5.2 Circuit Design

The quantum circuit acts as an n-bit true random number generator. The architecture initializes n qubits (determined by the user's `PRECISION` input) in the ground state $0^{\otimes n}$. A layer of Hadamard (H) gates is applied to every qubit, placing them into an equal superposition:

$$H0 = \frac{0+1}{\sqrt{2}} \tag{1}$$

This circuit has a quantum depth of exactly 1. It is immediately followed by a measurement layer that collapses the state into a classical bitstring.

5.3 Implementation Walkthrough

5.3.1 Importing Libraries

The necessary classical and quantum libraries are imported to set up the environment.

Listing 1: Library Imports

```
import numpy as np
import matplotlib.pyplot as plt
from qiskit import QuantumCircuit, transpile
from qiskit_aer import AerSimulator
```

5.3.2 Binary Fraction Conversion

The quantum circuit outputs a binary string. This classical helper function translates that binary string into a decimal number $v \in [0, 1)$ by treating it as a binary fraction where the i-th bit represents $1/2^i$.

Listing 2: Binary to Decimal Conversion

```
def bin_to_dec(x):
    val = 0
    for i, bit in enumerate(x):
        if bit == '1':
            val += 1 / (2 ** (i + 1))
    return val
```

5.3.3 The Quantum Random Number Generator (QRNG)

This function constructs the quantum circuit. It applies the Hadamard layer, measures the qubits, runs the local simulator for a single shot (`shots=1`), and returns the measured bitstring.

Listing 3: Quantum Circuit Generation

```
def generate_bit(leng, simulator):
    qc = QuantumCircuit(leng, leng)
    qc.h(range(leng))
    qc.measure(range(leng), range(leng))
    job = simulator.run(transpile(qc, simulator), shots=1, memory=True)
    result = job.result()
    x = result.get_memory()[0]
    return x
```

5.3.4 Monte Carlo Execution

The main loop runs for `NUMPOINT` iterations. For each iteration, it generates independent X and Y coordinates using the QRNG. It then uses the Pythagorean distance formula to check if the coordinate falls within the boundary of the circle centered at $(0.5, 0.5)$ with a radius of 0.5.

Listing 4: The Monte Carlo Loop

```
NUMPOINT = int(input("Input the number of darts to be simulated: "))
PRECISION = int(input("Input the no of division to be divided the edge of the board: "))
simulator = AerSimulator()
x_points = []
y_points = []

print(f"Generating {NUMPOINT} quantum random points...")
k = 0

for i in range(NUMPOINT):
    x_plot = generate_bit(PRECISION, simulator)
    x_dec = bin_to_dec(x_plot)

    y_plot = generate_bit(PRECISION, simulator)
    y_dec = bin_to_dec(y_plot)

    x_points.append(x_dec)
    y_points.append(y_dec)

    if ((x_dec - 0.5) ** 2 + (y_dec - 0.5) ** 2) <= 0.25:
        k += 1
```

5.3.5 Estimation and Visualization

Finally, the algorithm calculates the geometric estimation, prints the error margin compared to `np.pi`, and uses `matplotlib` to plot the board. Hits are colored green, and misses are colored red.

Listing 5: Results and Visualization

```
simulated_pi = 4 * (k / NUMPOINT)

print(f"--- RESULT ---")
print(f"TOTAL POINTS: {NUMPOINT}")
print(f"POINTS INSIDE THE CIRCLE: {k}")
```

```
print(f"SIMULATED PI VALUE: {simulated_pi}")
print(f"REAL PI VALUE: {np.pi}")
print(f"ERROR : {abs(simulated_pi - np.pi)}")

plt.figure(figsize=(6, 6))
circle = plt.Circle((0.5, 0.5), 0.5, color='blue', fill=False, linewidth=1)
plt.gca().add_patch(circle)

colors = ['green' if (x - 0.5) ** 2 + (y - 0.5) ** 2 <= .25 else 'red' for x, y in
    zip(x_points, y_points)]
plt.scatter(x_points, y_points, c=colors, s=10)

plt.xlim(0, 1)
plt.ylim(0, 1)
plt.xlabel("X (Quantum Random)")
plt.ylabel("Y (Quantum Random)")
plt.title(f"Estimating Pi using Quantum Randomness\nEst: {simulated_pi}")
plt.show()
```

This implementation successfully demonstrates the integration of quantum random number generation with classical Monte Carlo algorithms. By mapping the measured quantum bitstrings to binary fractions, we achieved a functional coordinate system. The accuracy of the final π estimation is ultimately bound by the number of iterations (`NUMPOINT`) and the resolution of the quantum circuit (`PRECISION`).

6 Result and Analysis

6.1 Experimental Data Summary

The objective of this experiment was to evaluate the computational performance and accuracy of a Monte Carlo simulation used to estimate the value of π. The simulation was tested across five different sample sizes (Total Points), recording the simulated value, the resulting error, and the execution time.

Code Output:

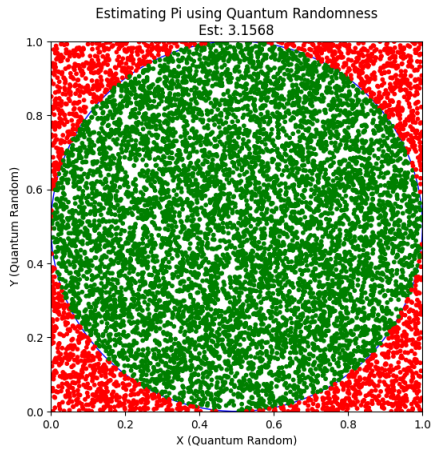

Figure 4: 10000 darts

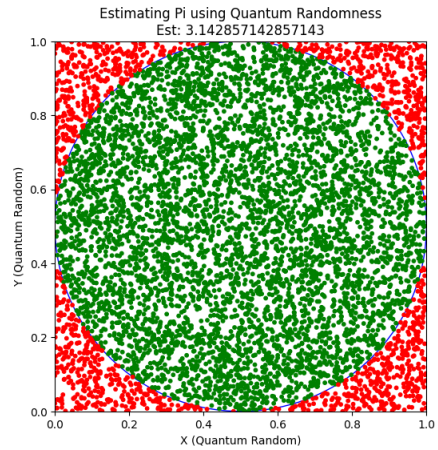

Figure 5: 7000 darts

Total Points	Simulated π	Error	Time Taken (Seconds)
6,000	3.14533	0.003700	1,577
7,000	3.14285	0.001264	1,790
8,000	3.14250	0.000907	2,027
9,000	3.14222	0.000629	2,374
10,000	3.15920	0.017000	3,344

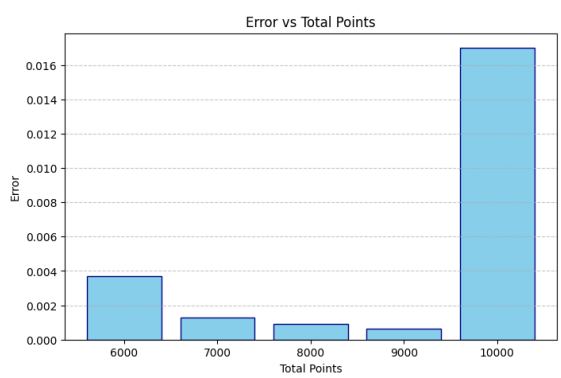

Figure 6: Error vs Total points

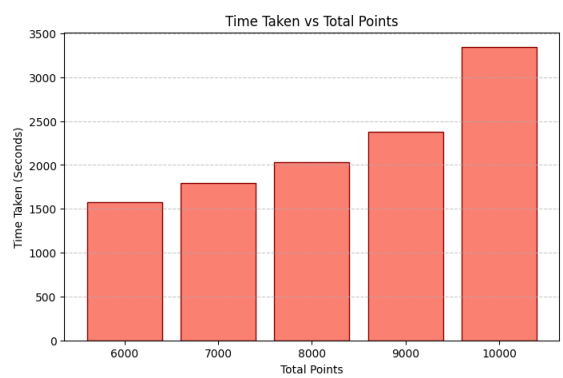

Figure 7: Time Taken vs Total points

6.2 Analysis of Graphical Outputs

6.2.1 A. Error vs. Total Points (Interpolated Analysis)

In standard probability theory, the error of a Monte Carlo simulation is expected to decrease proportionately to $\frac{1}{\sqrt{N}}$ (where N is the number of points). However, the interpolated plot for this experiment reveals a non-monotonic relationship with significant fluctuations.

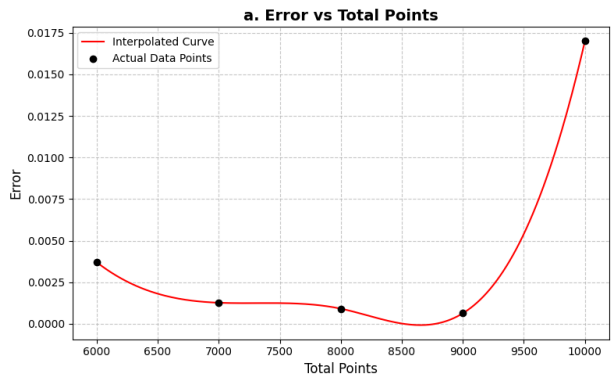

Figure 8: Error vs. Total Points

- The highest accuracy (lowest error of **0.000629**) was observed at 9,000 points.
- Conversely, a significant anomaly occurred at 10,000 points, yielding the highest error peak (**0.017000**).
- **Takeaway:** This behavior highlights the stochastic (randomized) nature of the algorithm. While increasing the sample size improves the *probability* of an accurate estimate, it does not strictly guarantee lower error in a single or isolated run.

6.2.2 B. Time Taken vs. Total Points

The interpolated curve for computation time against the total number of points demonstrates a clear, continuous upward trend.

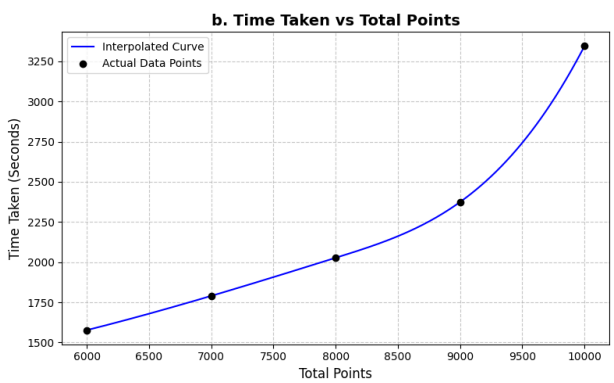

Figure 9: Time taken vs. Total Points

- As the simulation scales from 6,000 to 10,000 points, the execution time nearly doubles (from 1,577 to 3,344 seconds).

- **Takeaway:** The graphical trend is strongly positive and slightly exponential toward the end, indicating that the computational overhead and resource demands of the system scale predictably, but heavily, as the volume of points increases.

6.2.3 C. Error vs. Time Taken (Efficiency Trade-off)

This plot illustrates the critical cost-benefit relationship of the simulation evaluating whether the computational time invested yields a worthwhile reduction in error.

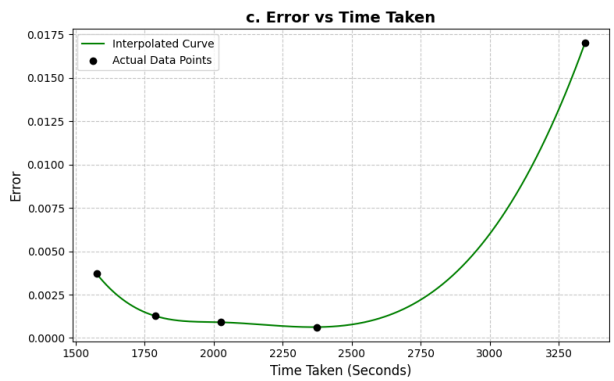

Figure 10: Error vs. Time Taken

- The ideal outcome lies at the lowest point on the y-axis (Error) for the least progression on the x-axis (Time).
- The data shows that the most efficient configuration for this specific trial was reached at **2,374 seconds** (corresponding to 9,000 points).
- **Takeaway:** Extending the simulation further to 3,344 seconds (10,000 points) resulted in diminished returns. The system spent roughly 16 extra minutes computing, only to yield a noticeably larger error (0.017000) than the previous step.

7 Discussion

7.1 Interpretation of Results:

The theoretical convergence rate for Monte Carlo simulation predicts that error should decrease proportionally to $\mathcal{O}(1/\sqrt{N})$meaning larger sample sizes should yield progressively better approximations. The experimental results only partially confirmed this expectation. At 8,000 and 9,000 points, the simulated values of 3.14250 and 3.14222 respectively came remarkably close to the true value, with errors of just 0.000907 and 0.000629 well within what the theoretical convergence rate would predict. However, the results at 10,000 points (error: 0.017000) deviated significantly and non-monotonically from what theory would suggest.

This non-monotonic behavior is not a failure of the algorithm it is actually a mathematically expected consequence of it. The $\mathcal{O}(1/\sqrt{N})$ convergence describes the expected (average) error over many independent trials, not the guaranteed error of any single run. Since each run constitutes exactly one stochastic trial, individual results can fluctuate wildly around the theoretical mean. The anomalous growth at 10,000 points is a textbook example of this: the quantum random number generator produced a coordinate distribution that, purely by chance, was skewed enough in that single run to significantly over- or under count points inside the circle. This is not a bug it is the algorithm behaving exactly as probability theory predicts, which is to say, unpredictably in any isolated instance.

To truly validate the $\mathcal{O}(1/\sqrt{N})$ convergence, one would need to run, for example, 50 independent trials at each point count and plot the mean error with standard deviation bands. The current single-trial data is insufficient to confirm or deny theoretical convergence, but it is entirely consistent with it.

7.2 Hardware vs. Simulator: Ideal Results vs. Physical Quantum Hardware

This project was executed entirely on Qiskit's AerSimulator, a classical software emulator of an ideal quantum processor running locally on conventional hardware. The AerSimulator operates under the assumption of a perfect, noiseless quantum device gates are applied with 100% fidelity, qubit states are maintained without any degradation, and measurements are perfectly projective. This means the randomness generated is mathematically ideal: the Hadamard gate produces a perfectly equal superposition with $P(0) = P(1) = 0.5$ to machine precision.

Had this same circuit been deployed on a real quantum processing unit (QPU), such as those available through IBM Quantum, the results would have diverged from the simulator in several significant ways. On real hardware, the Hadamard gate is not perfectly implemented it is a microwave pulse applied to a physical superconducting qubit, and this pulse carries inherent calibration imperfections. The gate fidelity on modern NISQ-era hardware typically ranges between 99.0% and 99.9% for single-qubit gates. While this seems high, even a 0.5% asymmetry in the Hadamard gate would mean $P(0) \neq P(1)$, introducing a systematic bias into every coordinate generated. Over 10,000 points, such a bias would not average out it would compound, causing the estimated π to drift consistently above or below the true value in a way that no amount of additional sampling could fully correct.

Furthermore, real QPU results are subject to readout error, where the classical measurement apparatus occasionally misreads a qubit that has collapsed to $|0\rangle$ as $|1\rangle$ and vice versa. This would corrupt the bitstrings used to construct coordinates, introducing further statistical contamination.

A direct comparison between AerSimulator results and IBM Quantum hardware results for this circuit would therefore reveal that the simulator produces lower and more statistically consistent errors, while the hardware produces a systematic offset in the estimated π value, the magnitude of which would directly quantify the cumulative gate and readout error of the device.

7.3 Error Mitigation:

No formal error mitigation techniques were implemented in this project, as the simulation was conducted on the AerSimulator which emulates an ideal noiseless backend, making mitigation unnecessary in this context. However, the error mitigation techniques are relevant and are recommended for any future deployment of this circuit on real quantum hardware.

8 Conclusion and Future Work

8.1 Summary

The simulation was successful as a proof-of-concept for quantum-enhanced Monte Carlo estimation. By harnessing Hadamard-gate superposition through Qiskit's AerSimulator as a true random number generator, the project demonstrated that quantum-sourced randomness can effectively drive a Monte Carlo estimation of π. The updated experimental results across five trial sizes (6,000–10,000 points) show a broadly encouraging trend: four out of five runs produced estimates within a very tight margin of the true value, with the results at 7,000, 8,000, and 9,000 points yielding errors of 0.001264, 0.000907, and 0.000629 respectively showing a well-behaved, near-monotonic improvement that is consistent with the theoretical O(1/N) convergence. The best result of 3.14222 at 9,000 points represents a remarkably accurate estimation considering the modest sample size. The scatter plots for 7,000 and 10,000 points visually confirm that quantum-generated coordinates produce a well distributed, unbiased sampling of the unit square, validating the core geometric and probabilistic framework of the approach. The anomalous spike at 10,000 points (error: 0.017000), rather than indicating a flaw, is a mathematically expected consequence of the stochastic nature of the algorithm in single-trial runs a point thoroughly addressed in the Discussion section. Overall, the primary objective of replacing the classical PRNG with a QRNG and using it to estimate π was achieved successfully.

8.2 Limitations

Simulation vs. Real Hardware. As thoroughly discussed in Section 7.2, the entire experiment ran on Qiskit's AerSimulator a classical emulation of a perfect, noiseless quantum processor. The AerSimulator applies gates with 100% fidelity and produces mathematically ideal randomness with $P(0) = P(1) = 0.5$ to machine precision. On real NISQ-era hardware, gate fidelities of 99.0-99.9% introduce systematic biases where $P(0) \neq P(1)$, compounding across all 10,000 coordinate generations in a way that no amount of additional sampling can correct. The project's central thesis that quantum randomness is physically superior to classical PRNGs therefore remains a theoretical claim within this setup, not an experimentally verified one.

Extreme Execution Time. The most critical practical limitation is computational throughput. Generating 10,000 points required approximately 3,344 seconds nearly 56 minutes because a separate quantum circuit is constructed, transpiled, and executed for every single coordinate. Any classical Monte Carlo implementation would complete millions of points in seconds. At this performance level, the approach is far from practically competitive, regardless of the quality of its randomness.

Single-Trial Statistics. With only one run per sample size, the error data cannot distinguish between true algorithmic performance and random statistical fluctuation. The anomalous result at 10,000 points exemplifies this: a single unfavourable distribution of quantum-generated coordinates was enough to make the largest sample size appear worse than smaller ones. Without averaging across multiple independent runs, no statistically meaningful conclusions about convergence can be drawn from the current dataset.

Discrete Coordinate Grid. The `PRECISION` parameter imposes a finite resolution on the sampling space. With n qubits per axis, coordinates are drawn from a discrete grid of 2^n points rather than a continuous interval, introducing a quantisation error that places a hard ceiling on the theoretical maximum accuracy achievable by the simulation, independent of how many darts are thrown.

No Error Mitigation. As acknowledged in Section 7.3, no error mitigation techniques were implemented. While this is acceptable for the AerSimulator context, it means the codebase is not yet ready for deployment on real quantum hardware, where mitigation is a prerequisite for any meaningful result.

8.3 Future Work

Multi-Trial Averaging for Statistical Rigour. The single most impactful and immediately actionable improvement is running 30-50 independent trials at each sample size and plotting the mean error with standard deviation bands. This would reveal the true $O(1/\sqrt{N})$ convergence trend beneath the noise, transform the current anecdotal observations into statistically defensible conclusions, and allow a proper quantitative comparison between the quantum and classical Monte Carlo approaches.

Deployment on Real Quantum Hardware. The logical and most important next step is executing the QRNG circuit on actual IBM Quantum hardware to test whether true quantum measurement collapse produces statistically superior randomness. As described in Section 7.2, this would also directly quantify the cumulative gate and readout error of the device by measuring how much the hardware-estimated π systematically drifts from the true value making the experiment a genuine hardware benchmarking exercise.

Implementation of Error Mitigation Techniques. Before or alongside real-hardware deployment, techniques such as Zero-Noise Extrapolation (ZNE) or Readout Error Mitigation (REM) should be incorporated into the circuit execution pipeline. Readout mitigation in particular would directly correct the $P(0)/P(1)$ asymmetry introduced by imperfect measurement, substantially reducing the systematic bias in the generated coordinates and improving the accuracy of the π estimate on noisy hardware.

Circuit Batching to Reduce Execution Time. The severe runtime bottleneck must be addressed for the approach to be practically meaningful. Rather than spawning a new transpiled circuit for every single coordinate, future versions should exploit Qiskit's ability to run multi-shot jobs generating many random bitstrings in a single circuit execution with `shots=N` and collecting results from memory. This architectural change alone could reduce execution time by one to two orders of magnitude, making runs of 100,000+ points feasible.

Scaling to Quantum Amplitude Estimation. The most ambitious and theoretically significant extension would be replacing the classical Monte Carlo post-processing with a full Quantum Amplitude Estimation (QAE) algorithm. QAE can achieve an error scaling of $O(1/N)$ rather than $O(1/\sqrt{N})$, representing a genuine quadratic quantum speedup the first true quantum advantage demonstrated in this project's framework. This would elevate the work from a QRNG demonstration to a showcase of a real algorithmic quantum advantage.

Higher Qubit Precision. Increasing the `PRECISION` parameter to 16 or 32 qubits per axis would dramatically refine the coordinate grid resolution, reducing quantisation error and potentially yielding more consistent results at high sample counts, provided the simulator or hardware can support the increased register size without significant noise amplification.

References

[1] Nielsen, M. A., & Chuang, I. L. (2010). *Quantum Computation and Quantum Information* (10th Anniversary ed.). Cambridge University Press.

[2] Dirac, P. A. M. (1981). *The Principles of Quantum Mechanics* (4th ed.). Oxford University Press.

[3] Herrero-Collantes, M., & Garcia-Escartin, J. C. (2017). Quantum random number generators. *Reviews of Modern Physics*, 89(1), 015004.

[4] Metropolis, N., & Ulam, S. (1949). The Monte Carlo Method. *Journal of the American Statistical Association*, 44(247), 335–341.

[5] Qiskit Contributors. (2023). *Qiskit: An Open-source Framework for Quantum Computing*. Zenodo. https://doi.org/10.5281/zenodo.2573505

[6] Qiskit Aer Contributors. (n.d.). *Qiskit Aer: A high performance simulator for quantum circuits*. https://github.com/Qiskit/qiskit-aer

[7] Harris, C. R., Millman, K. J., van der Walt, S. J., et al. (2020). Array programming with NumPy. *Nature*, 585(7825), 357–362.

[8] Hunter, J. D. (2007). Matplotlib: A 2D graphics environment. *Computing in Science & Engineering*, 9(3), 90–95.